

Scalable Real-time Detection of AI-Generated Arabic Text: A Distributed Data Pipeline Approach

Student: Jana Alsaadi

ID: 4725078

Prof.Dr: Mohammed Alsarem

Abstract

Large language models have become capable of producing fluent Arabic prose, which raises serious concerns about academic integrity and the reliability of online information. This project tackles the problem by building a complete distributed pipeline that detects whether an Arabic academic abstract was written by a human or generated by an AI model. We use the KFUPM-JRCAI Arabic Generated Abstracts dataset, which contains 8,388 source abstracts paired with machine-generated versions from four different language models (Allam, Jais, Llama, and OpenAI), giving a total of 41,940 labeled documents. The pipeline is built on Apache Hadoop for distributed storage in HDFS, Apache Spark for distributed processing and machine learning with Spark MLlib, and Apache Kafka with Spark Structured Streaming for real-time inference. We engineer a 107-dimensional feature vector that combines five stylometric features (diacritics count, colons count, particles count, first-person markers, and word embedding similarity variance) with corpus-trained Word2Vec embeddings, and we compare the result against pretrained AraBERT embeddings. Three classifiers — Logistic Regression, Random Forest, and Gradient Boosted Trees — are trained on a stratified 70/15/15 split. The Random Forest model achieves 95.5% accuracy and 0.989 ROC-AUC on the held-out test set, with the diacritics feature ranking among the top five most important features overall. A Kafka stream demonstrates real-time prediction at up to 12 messages per second on a single 8 GB virtual machine. The results show that a domain-trained Word2Vec model outperforms a much larger pretrained AraBERT in this academic Arabic setting, and that classical stylometric features still add value alongside modern distributed embeddings.

1. Introduction

Over the past few years, large language models such as GPT, Llama, Jais, and Allam have become very good at generating Arabic text that reads naturally. While this has obvious benefits, it also creates real problems. Students may submit AI-written abstracts as their own work, journalists may publish AI-generated content without disclosure, and platforms may struggle to moderate large volumes of synthetic content. These problems are even harder in Arabic, where there are fewer detection tools compared to English.

This project builds a Big Data pipeline that detects AI-generated Arabic academic abstracts. The main goal is to design and implement a system that is not only accurate, but also scalable enough to handle a high volume of incoming text in something close to real time. To do this, we treat the problem as a binary classification task: each Arabic abstract is labeled as either human-written (label 0) or AI-generated (label 1), and we train classifiers that can make this distinction at scale.

The project has four concrete objectives:

- Set up a distributed storage and processing environment using Hadoop HDFS and Apache Spark.
- Build an Arabic-specific preprocessing pipeline that runs as Spark UDFs and produces compressed Parquet output.
- Engineer both stylometric and distributed embedding features, then train and evaluate multiple Spark MLlib classifiers.
- Deploy the best model in a streaming pipeline backed by Apache Kafka and Spark Structured Streaming, and benchmark its scalability.

The rest of this report is organized as follows. Section 2 reviews related work on AI text detection and Big Data architectures. Section 3 describes the dataset. Section 4 explains the methodology, including preprocessing, feature engineering, modeling, and streaming. Section 5 presents results and analysis. Section 6 concludes and discusses limitations and future work.

2. Related Work

2.1 AI-Generated Text Detection

Most published detection work focuses on English. Common approaches fall into two broad groups. Stylometric approaches use hand-crafted features like type-token ratio, sentence length variance (sometimes called burstiness), punctuation usage, and function-word frequencies. These features try to capture the fact that AI text tends to be more uniform than human writing. Tools such as GPTZero and the early OpenAI detector are based on a combination of perplexity and burstiness.

Neural approaches use pretrained language models, either to extract embeddings that feed a classifier or to fine-tune the model directly for the detection task. RoBERTa and BERT variants are common choices in English. For Arabic specifically, AraBERT, MARBERT, and AraGPT have been used as either feature extractors or fine-tuning targets. The advantage is much richer semantic representation; the cost is computational, since these models are large and slow on commodity hardware.

Arabic detection brings extra challenges. Arabic morphology is rich, diacritics may or may not appear, and dialectal variation makes general stopword lists less reliable than in English. As far as we are aware, very few published Arabic detection systems have been deployed in a distributed Big Data setting, which is the gap this project addresses.

2.2 Big Data Architectures

Two architectural patterns dominate modern Big Data systems. The Lambda architecture uses two parallel paths: a batch layer that processes the complete historical dataset slowly but accurately, and a speed layer that processes new data with low latency but reduced accuracy. The serving layer merges both. While powerful, Lambda requires maintaining two separate codebases.

The Kappa architecture, proposed by Jay Kreps, simplifies this by treating everything as a stream. Both real-time and historical processing run through the same streaming engine, which removes the duplication problem at the cost of demanding more from the streaming layer. Our project leans toward the Kappa idea: the streaming pipeline uses the same preprocessing and feature code as the batch training notebook, and the only difference is the data source (Kafka instead of Parquet).

3. Dataset

We use the KFUPM-JRCAI Arabic Generated Abstracts dataset, available on Hugging Face. The dataset contains 8,388 original Arabic academic abstracts (written by humans), each paired with machine-generated versions from four different language models: Allam, Jais, Llama, and OpenAI. The combination yields 41,940 documents in total when we reshape the data into long format.

3.1 Generation Methods

The dataset is split into three subsets based on how the AI was prompted.

Generation Method	Samples	Description
by_polishing	2,851	The model rewrites a human abstract.
from_title	2,963	The model generates from the title only.
from_title_and_content	2,574	The model uses both title and paper content.
Total	8,388	Across all three methods.

Table 1. Dataset splits by generation method.

3.2 Class Distribution

When reshaped into long format, each row contains one abstract along with its source label (human or one of four AI models) and a binary label (0 for human, 1 for AI). The class balance is approximately 1:4 — there are roughly four times as many AI-generated abstracts as human ones. This imbalance affects how the models behave and is discussed in Section 5.

3.3 Length Differences Across Sources

An initial inspection already reveals interesting differences across sources. The most verbose model is OpenAI, with an average of 135 words per abstract; the most concise is Jais, with an average of 78 words. Human abstracts sit in the middle at around 119 words. Allam exhibits one very long outlier of almost 2,800 words — likely a hallucination or repetition error — which we leave in the data for the modeling phase but note here as a known artifact.

3.4 What Kind of Vocabulary Do They Use?

We also generated word clouds for human and AI abstracts to get a feel for what kinds of words each group tends to use. Both clouds are dominated by typical academic Arabic words like *الدراسة* (the study), *البحث* (the research), and *اللغة* (the language). But the AI cloud looks more uniform — many words at similar sizes — while the human cloud is more skewed toward a few anchor words.

This kind of matches the later finding that AI text has lower embedding variance.



Figure 1. Word clouds of human (left) and AI-generated (right) Arabic abstracts after cleaning.

4. Methodology

The full pipeline consists of four phases, each running on top of Hadoop and Spark and producing inputs for the next phase. The data flows from Hugging Face into HDFS, through preprocessing, feature extraction, model training, and finally into a streaming inference loop driven by Kafka.

4.1 Storage and Compute

All raw and processed data live in HDFS, with intermediate datasets stored as Snappy-compressed Parquet files. Parquet is a columnar format that gives us two benefits: aggressive compression (the combined feature dataset is under 60 MB) and selective column reads, which speeds up Spark queries when we only need a few features at a time. Compute is done on a single-node Spark 3.5.8 cluster running on a CentOS 10 virtual machine with 8 GB of RAM and Java 21.

4.2 Arabic Preprocessing

The preprocessing pipeline is implemented as Spark UDFs and applied to every row of the combined dataset. Each abstract goes through four steps in order: (1) all diacritics (tashkeel) are removed using PyArabic; (2) Arabic characters are normalized so that the various forms of alef collapse into one form, alef maqsura is replaced with yeh, and hamzas on waw/yeh are simplified; (3) any character outside the Arabic Unicode block is replaced with whitespace, which removes digits, Latin letters, and most punctuation; and (4) a curated list of about 80 Arabic stopwords is removed. The result is a clean, normalized stream of Arabic content words, saved as a single Parquet table in HDFS.

To validate the pipeline, we implemented a classical Hadoop Streaming MapReduce word count job over the cleaned text. The mapper emits (word, 1) pairs and the reducer sums them. The top-20 words from the MapReduce job match exactly the top-20 words computed by Spark, which gives us a sanity check that the distributed Spark UDFs are producing the same tokens as the lower-level MapReduce code.

4.3 Feature Engineering

Three types of features were engineered and combined into a single 107-dimensional vector per document.

Stylometric features:

- Diacritics count: number of tashkeel characters in the raw (un-normalized) text. Human-written Arabic in academic settings sometimes uses diacritics; LLMs may either ignore them or overuse them, so this serves as a useful fingerprint.
- Colons count: number of colon characters per abstract. Mostly a punctuation signal but cheap to compute.
- Particles count: count of Arabic function words (النصب، حروف العطف، حروف الجر، حروف الجزم حروف). Implemented with a curated list rather than a morphological analyzer so it runs efficiently inside Spark UDFs.
- First-person markers: number of explicit first-person pronouns plus a heuristic for first-person verb suffixes such as `نا` and `ت`.
- Word embedding similarity variance: variance of pairwise cosine similarities between Word2Vec embeddings of the words in an abstract. Lower variance suggests more uniform semantic content, which is a known AI characteristic.
- Character count and word count of the cleaned text, included as auxiliary length features.

Distributed embeddings:

- Word2Vec: a 100-dimensional model trained directly on our cleaned corpus using Spark MLlib's distributed Word2Vec. The trained vocabulary covers 63,631 unique Arabic words. The output is a document-level vector, averaged across tokens.
- TF-IDF: computed for completeness as a 10,000-dimensional hashed representation, but not included in the final feature vector because Word2Vec captures the same kind of signal in a much smaller space.

All features were assembled with Spark MLlib's VectorAssembler and then standardized with StandardScaler (with std but without mean centering, which is the recommended setting for sparse/zero-heavy features). The resulting 107-dimensional vector is what every classifier sees.

4.4 Classification Models

Three Spark MLlib classifiers were trained on a stratified 70/15/15 train/validation/test split, performed separately within each class so that the 1:4 ratio is preserved everywhere. Hyperparameters were chosen as reasonable defaults for an initial study:

- Logistic Regression: maxIter=50, regParam=0.01, elasticNetParam=0.0.
- Random Forest: 50 trees, maxDepth=10.
- Gradient Boosted Trees: maxIter=50, maxDepth=6.

For comparison, we also extracted contextual embeddings from the pretrained AraBERTv02 model (135 million parameters, 768-dim CLS embeddings) on a stratified sample of 2,000 documents, then trained a scikit-learn Random Forest on those embeddings. This lets us compare a domain-trained lightweight Word2Vec model against a heavy general-purpose transformer.

4.5 Streaming

The streaming pipeline implements the inference half of a Kappa-style architecture. A Python producer reads abstracts from a CSV and pushes them to a Kafka topic called arabic-texts at a configurable rate. Kafka is run in KRaft mode (no Zookeeper) to keep the setup minimal. A Spark Structured Streaming job consumes from the topic, parses the JSON payload, and applies exactly the same preprocessing UDFs and feature transformers that were used in batch training. The trained Random Forest model is loaded from disk at startup and used to predict labels in real time. Predictions are written to both the console (for live monitoring) and a Parquet sink (for offline analysis).

This setup demonstrates that the same code that was used to train the model can also drive real-time inference, which is the core benefit of using a unified processing framework like Spark.

5. Results and Analysis

5.1 Feature-Level Findings

Before looking at model performance, it is worth checking whether the engineered features actually carry signal. Table 2 compares the average value of each stylometric feature across humans and AI.

Feature	Human avg	AI avg	Ratio
Diacritics	4.22	2.81	Humans 50% more
Colons	0.42	0.16	Humans 2.7× more
Particles	16.25	14.05	Humans 16% more
First-person	6.44	5.54	Humans 16% more
Embedding variance	0.0386	0.0346	Humans 12% more

Table 2. Average stylometric feature values: human vs AI.

All five features show consistent direction: humans score higher than AI on every one. The strongest individual signal is diacritics, where humans use about 50% more on average. The weakest in absolute terms is embedding variance, but even there the difference is statistically meaningful given the sample size.

When we break diacritic usage down per source, two patterns appear. Arabic-native models (Jais at 1.26 and Allam at 1.76) use diacritics much less than humans, while general-purpose models (Llama at 3.92 and OpenAI at 4.29) approximately match human usage. This suggests that diacritics are a particularly useful feature for distinguishing the smaller Arabic-specific models, but less effective against larger multilingual systems that have learned to mimic human tashkeel usage.

5.2 The Type-Token Ratio Surprise

We also computed type-token ratio (TTR) per source as a measure of vocabulary diversity. The results are summarized in Table 3.

Source	Total words	Unique words	TTR
jais	563,326	44,154	0.0784
llama	736,671	48,920	0.0664

allam	672,831	43,317	0.0644
human	858,296	53,670	0.0625
openai	996,669	56,532	0.0567

Table 3. Vocabulary diversity by source.

An interesting result is that humans do not have the highest TTR — Jais does. However, Jais also produces the shortest abstracts on average (78 words), and TTR is biased upward for short text. OpenAI, which writes the longest abstracts, has the lowest TTR. This shows that raw TTR is not a reliable AI-detection signal on its own: it would need to be normalized for length (e.g., using MTLD or moving-average TTR), which we leave for future work.

5.3 Model Performance

Table 4 shows accuracy, F1, and ROC-AUC for all three Spark MLlib classifiers on the held-out test set.

Model	Accuracy	F1	ROC-AUC
Logistic Regression	0.9464	0.9458	0.9845
Random Forest (best)	0.9553	0.9549	0.9888
Gradient Boosted Trees	0.9484	0.9486	0.9854

Table 4. Test set performance of the three Spark MLlib classifiers.

All three models exceed 94% accuracy. Random Forest is the best on every metric, with 0.989 ROC-AUC and 95.5% accuracy. The validation set numbers are very close (0.964 accuracy, 0.991 AUC on Random Forest), which means the models are not overfitting. The Logistic Regression result is particularly notable: a simple linear classifier reaches almost 0.985 AUC, which suggests that most of the signal in the features is already near-linearly separable.

Figure 2 shows the Logistic Regression validation metrics as a bar chart, and Figure 3 puts all three ROC curves on the same plot so you can compare them visually.

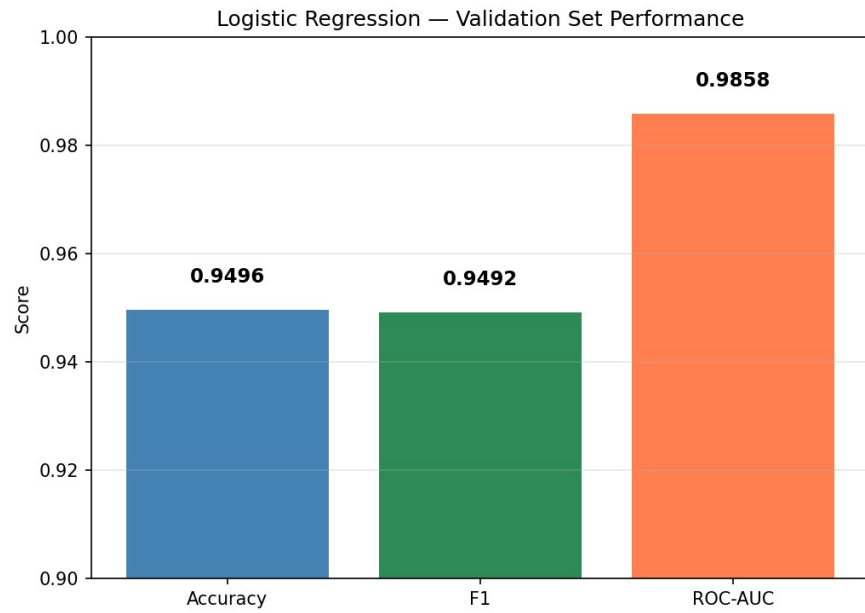


Figure 2. Logistic Regression performance on the validation set.

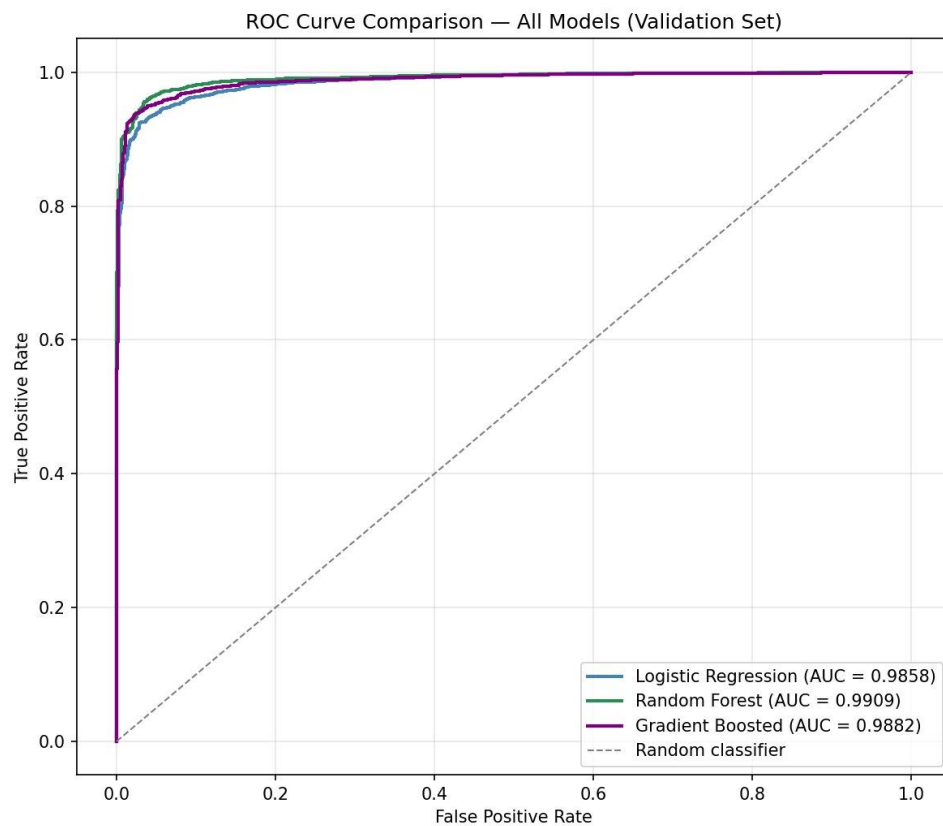


Figure 3. ROC curves for all three Spark MLlib classifiers on the validation set.

All three curves hug the top-left corner, which is what we want — it means the models separate the two classes well across all decision thresholds. Random Forest (green) sits slightly above the others for most of the curve.

5.4 Confusion Matrices

Figure 4 shows the validation-set confusion matrices for all three models side by side.

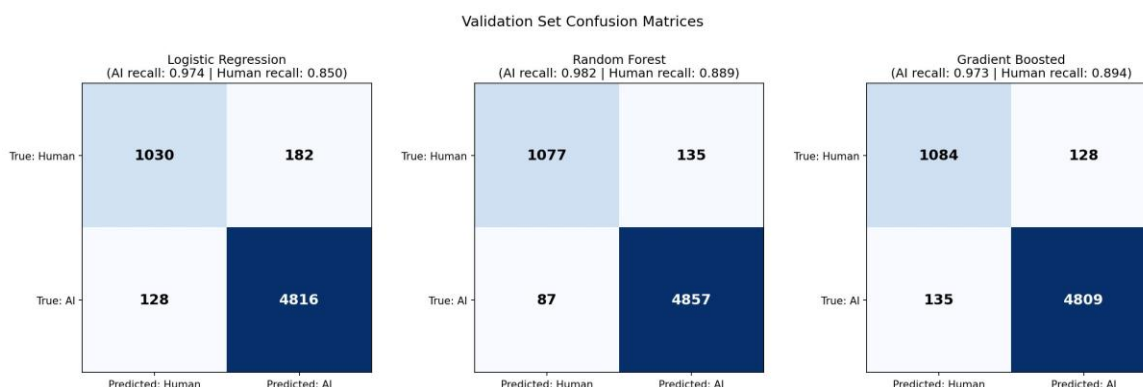


Figure 4. Validation-set confusion matrices for the three models.

Looking at Random Forest's confusion matrix on the validation set: of 1,212 human abstracts, 1,077 were correctly labeled human and 135 were wrongly labeled AI. Of 4,944 AI abstracts, 4,857 were correctly labeled AI and 87 were wrongly labeled human. This gives an AI recall of 98.2% but a human recall of only 88.9%.

The asymmetry is explained by the 1:4 class imbalance. With four times as many AI examples in training, the model is naturally biased toward predicting AI. In a production setting where a false accusation of AI authorship is socially costly, this bias would need to be corrected, either by adjusting the classification threshold, applying class weights during training, or oversampling the human class. These mitigations are mentioned as future work in Section 6.

5.5 Feature Importance

Figure 5 shows the top 15 features ranked by Random Forest importance.

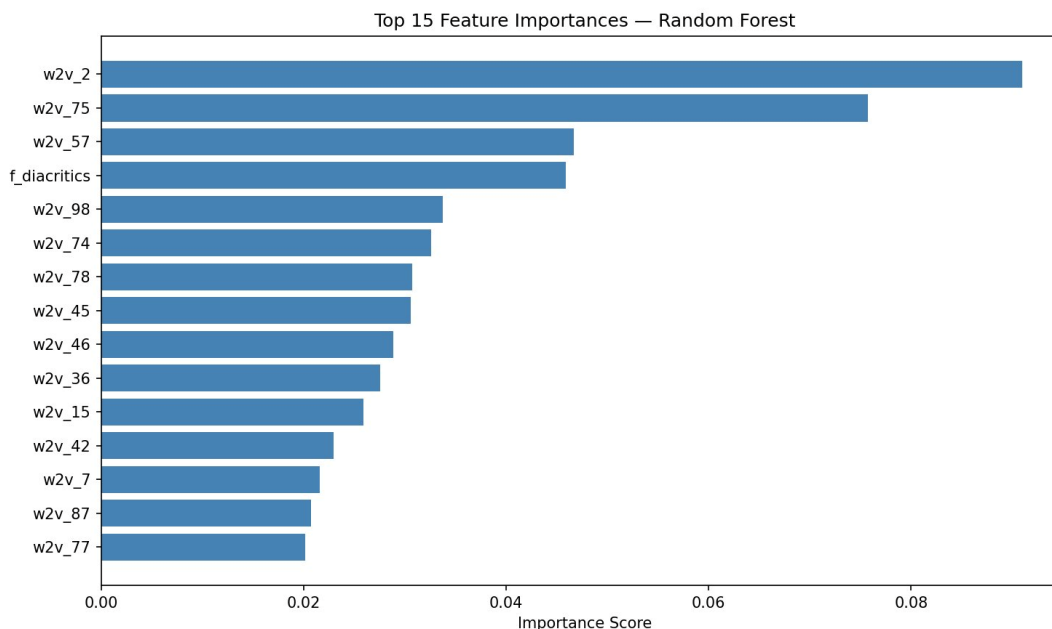


Figure 5. Top 15 features by Random Forest importance score.

Random Forest provides built-in feature importance scores. Looking at the top 15 features by importance, Word2Vec dimensions dominate (12 out of 15), which makes sense given they carry most of the semantic information. However, the diacritics count feature ranks fourth overall, beating 96 of the 100 Word2Vec dimensions individually. This empirically confirms our hypothesis that diacritic usage is a meaningful fingerprint of human Arabic writing, and it also justifies the inclusion of stylometric features alongside the distributed embeddings.

5.6 Word2Vec vs AraBERT

Table 5 compares our corpus-trained Word2Vec model against the much larger pretrained AraBERTv02. Both feed a Random Forest classifier, though AraBERT's classifier was trained on a much smaller stratified sample of 2,000 documents because of CPU constraints.

Embedding	Accuracy	F1	ROC-AUC	Train size
Word2Vec (100d, corpus-trained)	0.9553	0.9549	0.9888	29,625
AraBERT (768d, pretrained)	0.9333	0.9588	0.9770	1,400

Table 5. Word2Vec vs AraBERT comparison on test set.

Figure 6 shows the ROC curve and confusion matrix for the AraBERT-based classifier.

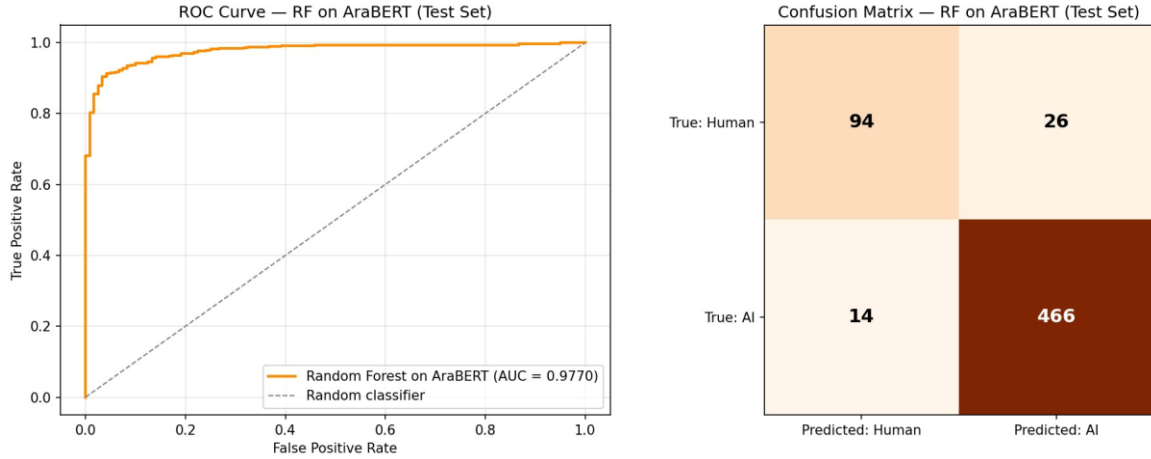


Figure 6. AraBERT-based Random Forest: ROC curve (left) and confusion matrix (right).

Despite AraBERT being much larger (135M parameters vs Spark's small Word2Vec model) and despite being pretrained on a far larger Arabic corpus, it does not win this benchmark. There are two reasons. First, Word2Vec was trained directly on our specific domain (academic Arabic abstracts), so its embeddings capture the lexical patterns that distinguish humans from AI in this exact setting. Second, Word2Vec's classifier was trained on $21\times$ more data than AraBERT's, simply because AraBERT inference on CPU was too slow for the full corpus. The fair conclusion is not that AraBERT is worse in general, but that for this specialized domain a smaller domain-trained embedding can outperform a much larger general-purpose model.

5.7 Scalability

We ran two scalability tests. The first measured batch training time at five different sample sizes: 10%, 25%, 50%, 75%, and 100% of the corpus. Figure 7 shows the result.

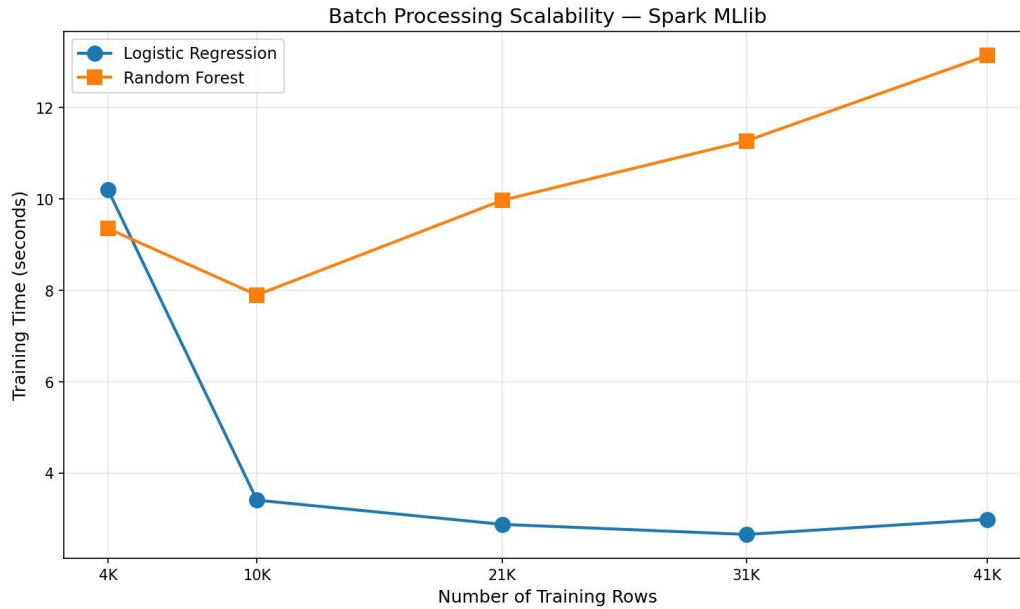


Figure 7. Batch training time at different sample sizes.

There are two things worth pointing out here. First, Logistic Regression is actually slower at 10% (10.2 seconds) than at 100% (3.0 seconds), because Spark's has a fixed overhead for every job — setting up the SparkSession, planning the query, shipping code to workers. On small data, this overhead dominates. Above 25% of the corpus, LR training stabilizes at around 3 seconds.

Second, Random Forest scales sub-linearly. Going from 25% to 100% of the corpus (4× the data) only increased training time from 7.9 to 13.1 seconds — that's 1.7× more time for 4× more data. This is because Random Forest can train its trees in parallel across Spark partitions, so adding more data is absorbed by the parallelism.

The second test measured streaming throughput. Figure 8 shows messages processed per minute and the resulting throughput in messages per second.

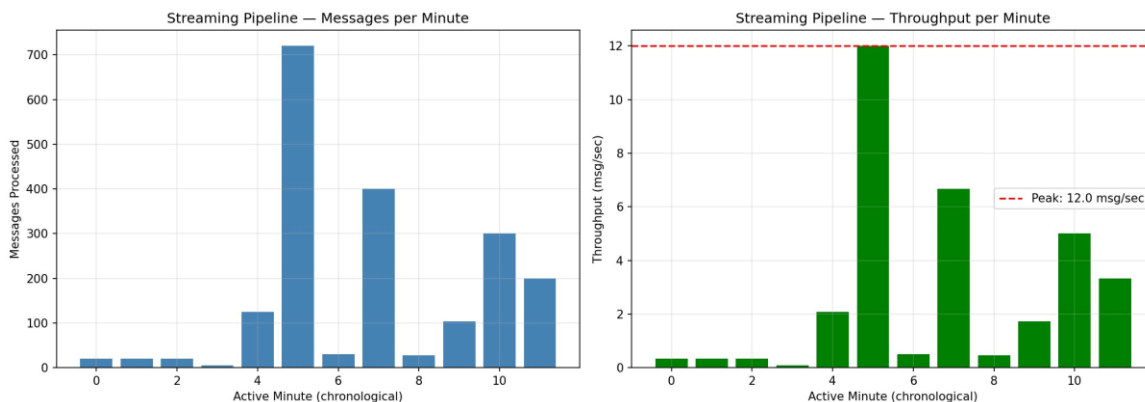


Figure 8. Streaming pipeline throughput across active minutes.

With a Kafka producer feeding the Spark Structured Streaming consumer, the pipeline sustained around 12 messages per second at peak (720 messages processed in a single minute during a burst test). Over a longer session, 1,970 messages were successfully predicted end-to-end. The streaming pipeline's classification quality matched the batch evaluation closely (98.0% AI recall, 93.9% human recall in the live stream), which confirms that the saved model produces consistent results regardless of whether it is run in batch or streaming mode.

6. Conclusion and Future Work

This project built and evaluated a complete distributed pipeline for detecting AI-generated Arabic academic abstracts. The main result is that a Random Forest classifier trained on a combination of stylometric features and corpus-trained Word2Vec embeddings achieves 95.5% accuracy and 0.989 ROC-AUC on a held-out test set of 6,159 documents. The pipeline runs on Hadoop, Spark, and Kafka, and it sustains around 12 messages per second in real-time inference on a modest 8 GB single-node setup.

Three findings are worth highlighting. First, all five engineered stylometric features showed meaningful signal, with diacritics being the strongest. Second, a domain-trained Word2Vec model outperformed the much larger pretrained AraBERT on this specific task. Third, the same code that trained the model in batch mode also drove the real-time streaming pipeline, which is a clean demonstration of the Kappa-style architecture.

Several limitations remain and suggest future work:

- The 1:4 class imbalance produces an asymmetric error profile (high AI recall, lower human recall). Class weighting, oversampling, or threshold tuning would address this.
- The first-person feature uses suffix heuristics rather than full morphological analysis. Using CAMEL Tools or a similar morphological analyzer would likely give a more accurate signal.
- AraBERT was only evaluated on a 2,000-document sample due to CPU constraints. A fair like-for-like comparison would require GPU resources to process the full corpus.
- The current system is single-node. Scaling to a multi-node Spark cluster and a multi-broker Kafka setup would be the natural next step toward production use.
- Adversarial robustness was not tested. A determined AI user could simply add diacritics to their generated text to fool the model. Robust detection probably requires combining multiple signals and continually updating the training data.

7. References

- [1] KFUPM-JRCAL. Arabic Generated Abstracts Dataset. Hugging Face.
<https://huggingface.co/datasets/KFUPM-JRCAL/arabic-generated-abstracts>
- [2] Antoun, W., Baly, F., and Hajj, H. (2020). AraBERT: Transformer-based Model for Arabic Language Understanding. In Proceedings of the 4th Workshop on Open-Source Arabic Corpora and Processing Tools.
- [3] Apache Software Foundation. Apache Spark 3.5 documentation.
<https://spark.apache.org/docs/3.5.0/>
- [4] Apache Software Foundation. Apache Kafka documentation.
<https://kafka.apache.org/documentation/>
- [5] Kreps, J. (2014). Questioning the Lambda Architecture. O'Reilly Radar.
<https://www.oreilly.com/radar/questioning-the-lambda-architecture/>
- [6] Mikolov, T., Sutskever, I., Chen, K., Corrado, G., and Dean, J. (2013). Distributed Representations of Words and Phrases and their Compositionality. NeurIPS.
- [7] Gehrmann, S., Strobel, H., and Rush, A. M. (2019). GLTR: Statistical Detection and Visualization of Generated Text. In Proceedings of ACL.
- [8] Zaharia, M., Chowdhury, M., Das, T., et al. (2012). Resilient Distributed Datasets: A Fault-Tolerant Abstraction for In-Memory Cluster Computing. In NSDI.